

# Implementación desde cero de kNN para clasificación y recomendaciones sobre trabajos del cluster Kabre

Proyecto 1 - MC-8851 Optimización y Programación Numérica

Daniel Garbanzo      Luis Carlos Navarro Todd  
Ricardo Shum Estrada

2026-04-21

## Tabla de contenidos

|          |                                                                           |           |
|----------|---------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>1. Introducción</b>                                                    | <b>2</b>  |
| <b>2</b> | <b>2. Marco teórico</b>                                                   | <b>3</b>  |
| 2.1      | 2.1 k-Nearest Neighbors como clasificador basado en instancias . . . . .  | 3         |
| 2.2      | 2.2 Normalización y construcción del espacio de características . . . . . | 4         |
| 2.3      | 2.3 Distancias L2 y L1 . . . . .                                          | 4         |
| 2.4      | 2.4 Recomendaciones contrafactuales y recourse . . . . .                  | 4         |
| <b>3</b> | <b>3. Descripción del dataset</b>                                         | <b>5</b>  |
| 3.1      | 3.1 Origen y contexto real . . . . .                                      | 5         |
| 3.2      | 3.2 Limpieza y preparación inicial . . . . .                              | 5         |
| 3.3      | 3.3 Variable objetivo . . . . .                                           | 6         |
| 3.4      | 3.4 Hallazgos principales del análisis exploratorio . . . . .             | 7         |
| <b>4</b> | <b>4. Implementación</b>                                                  | <b>9</b>  |
| 4.1      | 4.1 Construcción de la matriz de características . . . . .                | 9         |
| 4.2      | 4.2 División del dataset y normalización . . . . .                        | 10        |
| 4.3      | 4.3 Implementación desde cero del algoritmo kNN . . . . .                 | 10        |
| 4.4      | 4.4 Explicabilidad y recomendaciones . . . . .                            | 10        |
| 4.5      | 4.5 Comentarios de diseño . . . . .                                       | 11        |
| 4.6      | 4.6 Pseudocódigo del pipeline principal . . . . .                         | 11        |
| 4.7      | 4.7 Fragmentos clave con anotaciones . . . . .                            | 11        |
| 4.7.1    | 4.7.1 Construcción del vector de características . . . . .                | 11        |
| 4.7.2    | 4.7.2 Distancia euclidiana por batch . . . . .                            | 12        |
| 4.7.3    | 4.7.3 Selección de vecinos y decisión final . . . . .                     | 12        |
| <b>5</b> | <b>5. Resultados y análisis</b>                                           | <b>12</b> |
| 5.1      | 5.1 Selección de k . . . . .                                              | 12        |

|          |                                                               |           |
|----------|---------------------------------------------------------------|-----------|
| 5.2      | 5.2 Evaluación final del clasificador . . . . .               | 13        |
| 5.3      | 5.3 Análisis de recomendaciones contrafactuales . . . . .     | 14        |
| 5.4      | 5.4 Variables más frecuentes en las recomendaciones . . . . . | 15        |
| 5.5      | 5.5 Análisis crítico . . . . .                                | 16        |
| 5.6      | 5.6 Comparación con baseline . . . . .                        | 16        |
| <b>6</b> | <b>6. Conceptos del curso aplicados</b>                       | <b>16</b> |
| 6.1      | 6.1 Vectores de características . . . . .                     | 16        |
| 6.2      | 6.2 Normalización de datos . . . . .                          | 17        |
| 6.3      | 6.3 Norma L2 . . . . .                                        | 17        |
| 6.4      | 6.4 Norma L1 . . . . .                                        | 17        |
| 6.5      | 6.5 Combinaciones lineales en la votación ponderada . . . . . | 18        |
| <b>7</b> | <b>7. Conclusiones</b>                                        | <b>18</b> |
| <b>8</b> | <b>8. Referencias</b>                                         | <b>18</b> |

## Integrantes y carnes

| Integrante               | Carne      |
|--------------------------|------------|
| Daniel Garbanzo          | 2022117129 |
| Luis Carlos Navarro Todd | 2022212158 |
| Ricardo Shum Estrada     | 2017144193 |

**Algoritmo:** k-Nearest Neighbors (kNN) desde cero

**Dataset:** Historial de trabajos del cluster Kabre de CeNAT/CONARE

**Fuente del dataset:** `dataset.csv` del repositorio Project-1-CeNAT

**Entregable base:** `kabre_jobs_xai.ipynb`

## 1. 1. Introducción

Los clusters de cómputo de alto rendimiento (HPC) son infraestructuras compartidas donde muchos usuarios envían trabajos con requerimientos variables de CPUs, memoria, nodos, prioridad, tiempo límite y colas de ejecución. En este entorno, una configuración inadecuada no solo incrementa la probabilidad de fallo, sino que también consume recursos de computo, tiempo de cola y esfuerzo de investigación sin producir un resultado útil. En Kabre, operado por CeNAT/CONARE, este problema tiene relevancia directa porque el historial de ejecuciones muestra que una proporción considerable de jobs no termina exitosamente.

El objetivo de este proyecto fue construir, desde cero, un sistema basado en k-Nearest Neighbors (kNN) capaz de responder dos preguntas prácticas: primero, si un trabajo nuevo tiene alta o baja probabilidad de completarse exitosamente; segundo, que cambios en su configuración podrían aumentar esa probabilidad. La pregunta analítica que guió el trabajo fue la siguiente: **“Es posible predecir con precisión razonable si un trabajo en el cluster Kabre se completara exitosamente y, si no, que cambios específicos en su configuración podrían aumentar esa probabilidad?”**

Se eligió kNN por tres razones principales. En primer lugar, permite una implementación completamente desde cero usando operaciones elementales sobre vectores y distancias. En segundo lugar, su interpretación geométrica es natural: la predicción de un punto depende de trabajos historicamente similares en el espacio de características. En tercer lugar, esta misma geometría favorece la construcción de explicaciones y recomendaciones contrafactuales, porque es posible identificar que variables acercan o alejan un job de ejemplos que si terminaron como COMPLETED.

El entregable principal del proyecto se organizo bajo la metodología CRISP-DM. En el notebook se desarrollaron las fases de entendimiento del negocio, entendimiento de los datos, preparacion, modelado, evaluación y conclusiones. El informe sintetiza ese proceso con énfasis en la implementación algebraica del modelo, los hallazgos del análisis exploratorio, el desempeño del clasificador y la interpretación critica de sus recomendaciones.

## 2. Marco teórico

### 2.1. 2.1 k-Nearest Neighbors como clasificador basado en instancias

El algoritmo kNN clasifica una observación nueva a partir de las etiquetas de las  $k$  observaciones más cercanas del conjunto de entrenamiento. A diferencia de otros modelos, no construye una función paramétrica explícita durante el entrenamiento; en su forma más simple, conserva los datos ya procesados y decide por vecindad local. Esta propiedad lo convierte en un método de *lazy learning* o aprendizaje perezoso y hace que su comportamiento dependa fuertemente del espacio de características definido para el problema (Cover & Hart, 1967; Hastie, Tibshirani, & Friedman, 2009).

Sea un vector de entrada  $x \in \mathbb{R}^d$  y un conjunto de entrenamiento  $\{(x_i, y_i)\}_{i=1}^n$ . El clasificador busca los  $k$  vecinos más cercanos de  $x$  según una métrica de distancia y luego asigna la clase más frecuente entre ellos:

$$\hat{y}(x) = \text{mode}\{y_{(1)}, y_{(2)}, \dots, y_{(k)}\}$$

donde  $y_{(j)}$  representa la etiqueta del vecino  $j$ -ésimo más cercano.

En notación matricial, el conjunto procesado puede verse como una matriz de datos

$$X \in \mathbb{R}^{n \times d}$$

y un vector de etiquetas binarias

$$y \in \{0, 1\}^n$$

donde en este trabajo se obtiene finalmente una matriz aproximada de tamaño  $42745 \times 15$ .

## 2.2. 2.2 Normalización y construcción del espacio de características

En kNN, la cercanía entre puntos solo tiene sentido si las variables se encuentran en escalas comparables. Si una feature posee magnitudes mucho mayores que otra, dominara la distancia total aunque su importancia real no sea superior. Por esta razón, el proyecto aplico normalización min-max a las variables numericas:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

La normalización transforma cada variable al intervalo  $[0, 1]$  y evita que columnas como memoria, prioridad o tiempo límite distorsionen el calculo de similitud. Además, las variables categóricas se representaron mediante *one-hot encoding*, lo que permite incorporarlas como dimensiones independientes dentro del vector de entrada (Han, Kamber, & Pei, 2012).

## 2.3. 2.3 Distancias L2 y L1

El proyecto implementa dos distancias desde cero. La principal para el modelo final fue la distancia euclidiana o norma L2:

$$\|x - z\|_2 = \sqrt{\sum_{j=1}^d (x_j - z_j)^2}$$

Esta distancia penaliza con mayor fuerza diferencias grandes en una o varias coordenadas y es una opcion natural cuando las features ya fueron normalizadas.

Tambien se implementó la distancia Manhattan o norma L1:

$$\|x - z\|_1 = \sum_{j=1}^d |x_j - z_j|$$

La distancia L1 puede ser util cuando se desea una medida menos sensible a grandes discrepancias puntuales. Aunque el modelo final del notebook usa L2, la implementacion de L1 forma parte del analisis algebraico exigido por el curso y deja abierta la puerta a comparaciones futuras.

## 2.4. 2.4 Recomendaciones contrafactuales y recourse

Una predicción negativa es más útil si puede acompañarse de una recomendación accionable. En este proyecto se adopta la idea de *algorithmic recourse*: generar cambios en las variables de entrada que acerquen una observacion a la region del espacio asociada con la clase favorable (Karimi, Barthe, Scholkopf, & Valera, 2022). En lugar de solo indicar “este trabajo podria fallar”, el sistema intenta sugerir modificaciones concretas en memoria, prioridad, nodos o colas de ejecución para aumentar la probabilidad predicha de COMPLETED.

La propuesta implementada en el notebook usa una búsqueda codiciosa sobre features modificables. En cada iteración se evalua cual cambio incremental produce la mayor mejora en la probabilidad

predicha de la clase objetivo. Aunque esto no produce garantías causales, si ofrece una forma interpretable de orientar decisiones operativas antes del envío del job.

### 3. Descripción del dataset

#### 3.1. Origen y contexto real

El dataset corresponde a trabajos reales ejecutados en el cluster Kabre de CeNAT/CONARE durante 2024. Los registros provienen del historial del scheduler Slurm, sistema de administración y asignación de recursos ampliamente utilizado en entornos HPC (Yoo, Jette, & Grondona, 2003). El uso de un dataset operativo da al proyecto una relevancia practica clara, pues las predicciones se formulan sobre decisiones reales de uso del cluster.

En su estado bruto, el dataset contiene **46,744** filas y **12** columnas principales. Entre ellas se encuentran variables numericas como `ReqCPUS`, `ReqNodes`, `ResvCPURAW`, `Priority` y `CPUTimeRAW`, asi como variables categóricas y mixtas como `Partition`, `QOS`, `State`, `ReqMem` y `TimelimitRaw`. La fecha de envío se registra en la columna `Submit`, que permite derivar patrones temporales.

#### 3.2. Limpieza y preparación inicial

El notebook registra **3,249** valores nulos totales y **824** duplicados antes de la limpieza. Tras eliminar nulos y registros duplicados, el conjunto queda en **42,745** filas sin datos faltantes ni repetidos. Este paso fue necesario para garantizar consistencia en el calculo de distancias, ya que kNN no tolera adecuadamente registros incompletos sin un tratamiento explícito.

También se detectaron dos necesidades de transformación específicas:

1. `TimelimitRaw` contiene valores no numéricos en algunos casos, por lo que se construyó `TimelimitSec` a partir de reglas asociadas a la partición.
2. Las particiones originales presentan alta cardinalidad y fuertes diferencias de comportamiento, por lo que se agruparon en cinco familias principales: `Andalan`, `Nu`, `Dribe`, `Nukwa` y `Kura`.

La siguiente tabla resume las variables más relevantes y su tratamiento en el pipeline:

| Variable                  | Tipo original | Tratamiento en el notebook                                    | Uso final           |
|---------------------------|---------------|---------------------------------------------------------------|---------------------|
| <code>ReqCPUS</code>      | Numerica      | Se conserva                                                   | Feature numerica    |
| <code>ReqMem</code>       | Cadena        | Se convierte a <code>ReqMem_MB</code>                         | Feature numerica    |
| <code>ReqNodes</code>     | Numerica      | Se conserva                                                   | Feature numerica    |
| <code>ResvCPURAW</code>   | Numerica      | Se conserva                                                   | Feature numerica    |
| <code>TimelimitRaw</code> | Mixta         | Se transforma a <code>TimelimitSec</code>                     | Feature numerica    |
| <code>Priority</code>     | Numerica      | Se conserva                                                   | Feature numerica    |
| <code>Submit</code>       | Fecha-hora    | Se derivan <code>submit_hour</code> y <code>submit_dow</code> | Features temporales |

| Variable         | Tipo original | Tratamiento en el notebook                       | Uso final         |
|------------------|---------------|--------------------------------------------------|-------------------|
| <b>Partition</b> | Categorica    | Se agrupa como <b>PartitionGroup</b>             | One-hot encoding  |
| <b>QOS</b>       | Categorica    | Se conserva                                      | One-hot encoding  |
| <b>State</b>     | Categorica    | Se binariza en <b>COMPLETED</b> vs no completado | Variable objetivo |

Tambien es útil resumir los estadísticos globales más importantes del conjunto de trabajo:

| Indicador                                               | Valor  |
|---------------------------------------------------------|--------|
| Registros brutos                                        | 46,744 |
| Columnas base del dataset                               | 12     |
| Columnas del dataframe de trabajo tras transformaciones | 16     |
| Nulos originales                                        | 3,249  |
| Duplicados originales                                   | 824    |
| Registros finales utilizables                           | 42,745 |

### 3.3. 3.3 Variable objetivo

La columna **State** describe el estado final del trabajo. Para el problema de clasificación se transformó en una etiqueta binaria:

- 1 si **State** == **COMPLETED**
- 0 para **FAILED**, **CANCELLED** o **TIMEOUT**

La distribución observada en el notebook fue la siguiente:

| Estado           | Trabajos | Porcentaje |
|------------------|----------|------------|
| <b>FAILED</b>    | 24,335   | 52.06 %    |
| <b>COMPLETED</b> | 16,782   | 35.90 %    |
| <b>CANCELLED</b> | 4,001    | 8.56 %     |
| <b>TIMEOUT</b>   | 1,621    | 3.47 %     |

Este desbalance justifica el uso de métricas adicionales a **accuracy**, ya que un clasificador ingenuo que siempre prediga “no completado” alcanzaría aproximadamente **64.10 %** de precisión global solo por la distribución de clases.

### 3.4. 3.4 Hallazgos principales del análisis exploratorio

El EDA del notebook deja varios hallazgos relevantes:

- Solo alrededor del 36 % de los trabajos terminan como **COMPLETED**, por lo que el problema tiene una clase positiva minoritaria.
- La tasa de éxito cambia de forma importante entre particiones, lo que confirma que **Partition** es una feature informativa.
- Existen patrones temporales por hora del día, lo cual justifica incluir **submit\_hour** y **submit\_dow** en el vector de entrada.
- La columna **TimelimitRaw** requiere tratamiento especial y no puede usarse directamente en todos los casos.
- Aunque existen outliers en variables como memoria, tiempo y prioridad, estos no se eliminaron porque reflejan comportamientos plausibles dentro de un entorno HPC real.

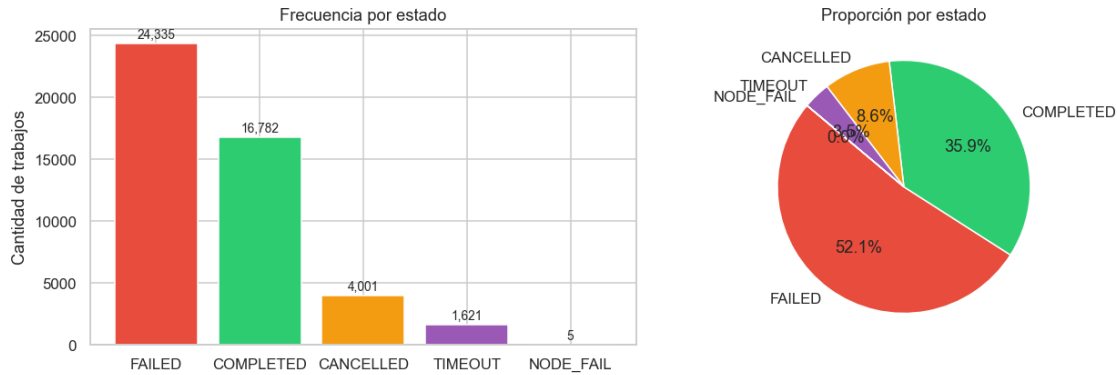


Figura 1: Distribucion de la variable objetivo **State** mediante frecuencias absolutas y proporciones. La figura evidencia que **FAILED** domina el conjunto y que **COMPLETED** representa solo alrededor del 36 % de los casos.

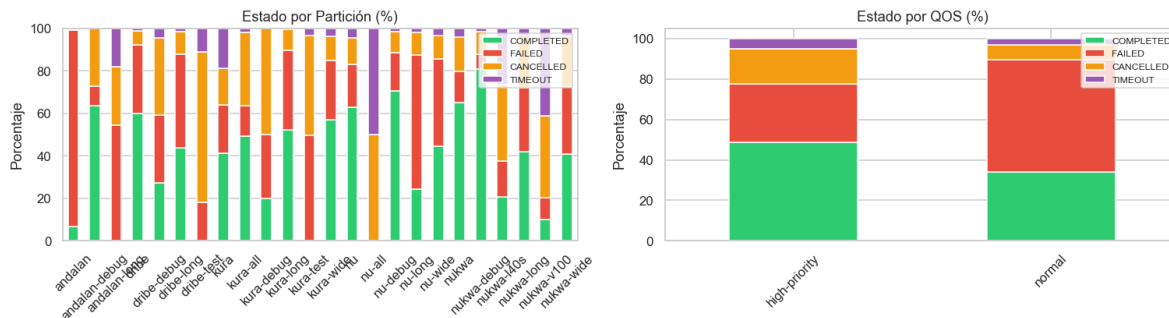


Figura 2: Comparacion de la proporcion de estados por **Partition** y **QOS**. Esta figura confirma que la particion y la política de servicio influyen en la tasa de éxito, por lo que ambas variables deben formar parte del vector de entrada.

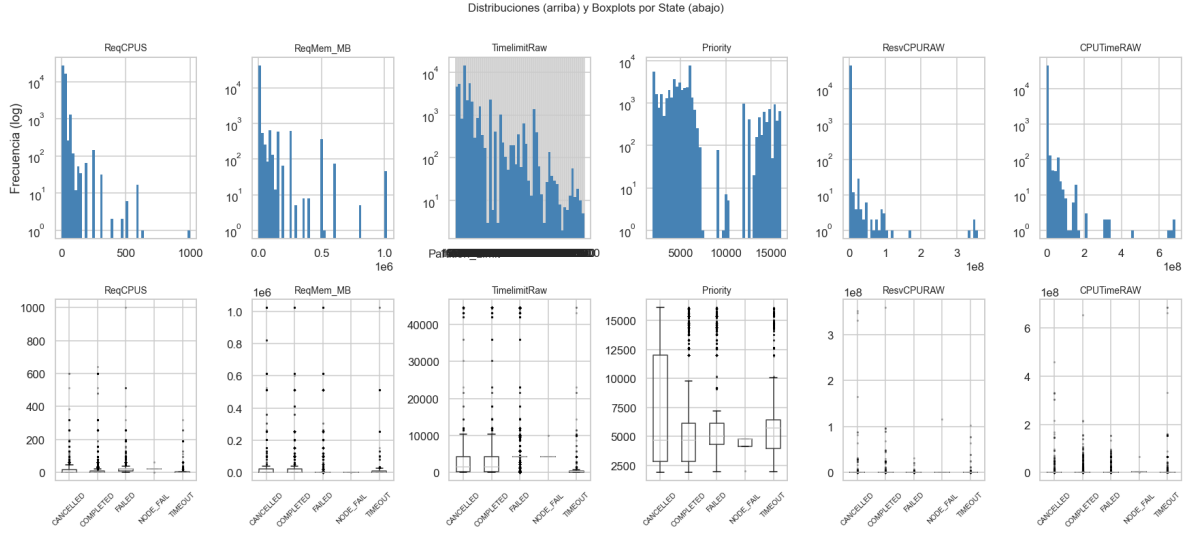


Figura 3: Distribuciones numericas y boxplots por estado. La figura muestra dispersion, asimetrias y presencia de outliers en variables clave del scheduler.



Figura 4: Analisis temporal de envios por mes y por hora del dia. El patron observado respalda la inclusion de variables temporales derivadas de **Submit**.



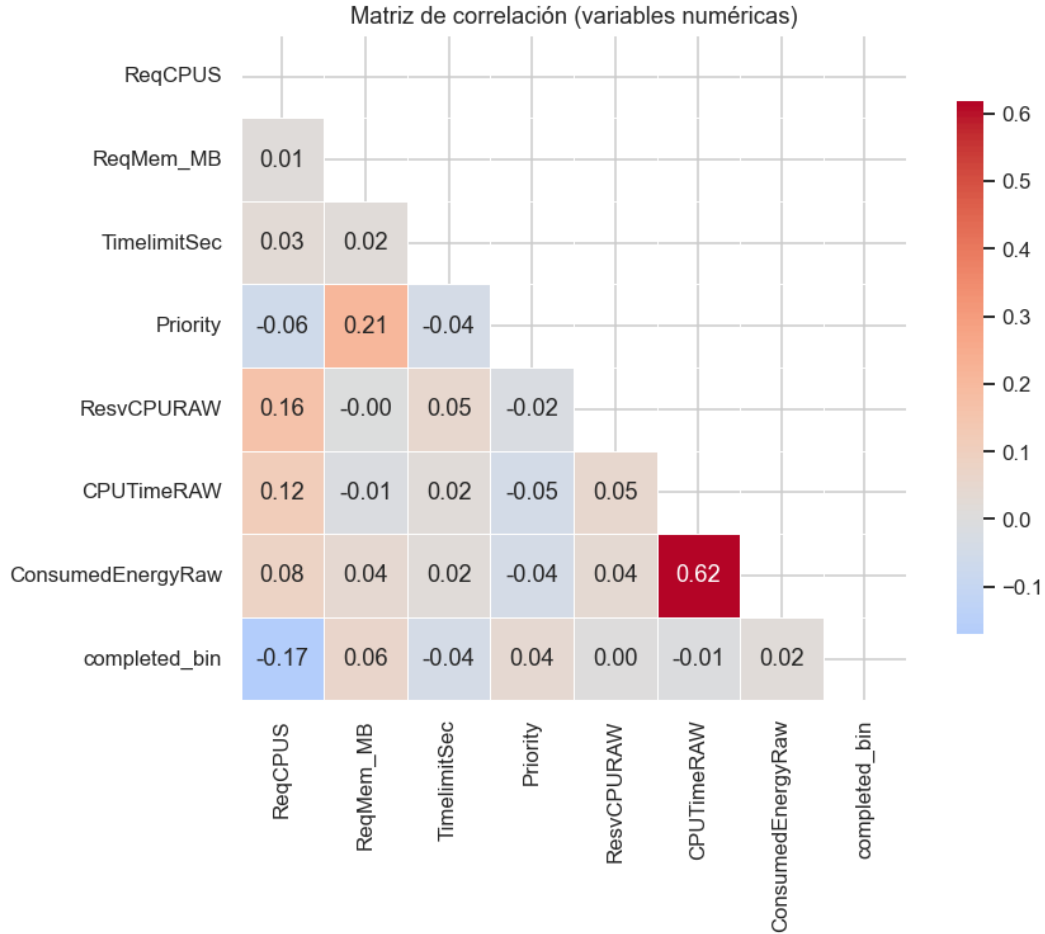


Figura 5: Mapa de calor de correlaciones entre variables numéricas y la variable binaria `completed_bin`. Sirve para identificar relaciones lineales aproximadas y posibles señales predictivas.

## 4. Implementación

### 4.1. Construcción de la matriz de características

Después de la limpieza, el notebook construye una matriz **X** con **42,745** muestras utilizables y una etiqueta binaria y del mismo tamaño. La implementación actual representa cada job mediante un vector de **15 dimensiones efectivas**:

- 8 variables numéricas: ReqCPUS, ReqMem\_MB, ReqNodes, ResvCPURAW, TimelimitSec, Priority, submit\_hour, submit\_dow
- 5 columnas one-hot para PartitionGroup
- 2 columnas one-hot para QOS

Este paso materializa el concepto de vector de características del curso y permite aplicar operaciones algebraicas y métricas de distancia de manera uniforme.

## 4.2. 4.2 División del dataset y normalización

El notebook divide los datos con un esquema estratificado de entrenamiento y prueba usando una proporción **80/20**, preservando la distribución aproximada de clases entre ambos subconjuntos. Posteriormente se ajusta un escalador min-max sobre el conjunto de entrenamiento y se transforma tanto **X\_train** como **X\_test** al rango  $[0, 1]$ .

La normalización es esencial para kNN, porque el algoritmo compara puntos directamente en el espacio de características. Sin este paso, variables de gran magnitud como memoria o prioridad dominarían la similitud total.

## 4.3. 4.3 Implementación desde cero del algoritmo kNN

La clase **KNNClassifier** del notebook se implementa sin usar `sklearn.fit()` ni equivalentes. La lógica central incluye:

1. almacenamiento del conjunto de entrenamiento,
2. calculo vectorizado de distancias L2 o L1,
3. seleccion de los **k** vecinos mas cercanos mediante `argpartition`,
4. votacion mayoritaria para la clase final,
5. estimacion de probabilidades por proporcion de vecinos.

La implementación también contempla una variante de vecinos ponderados mediante pesos inversamente proporcionales a la distancia:

$$v_i = \frac{1}{d_i + \varepsilon}$$

donde  $\varepsilon = 10^{-9}$  evita divisiones por cero. Aunque el modelo final se ejecuto con `weighted=False`, esta extensión muestra como el problema puede interpretarse en terminos de combinaciones lineales de etiquetas vecinas.

## 4.4. 4.4 Explicabilidad y recomendaciones

Además de clasificar, el notebook implementa tres mecanismos de interpretación:

1. **Vector decoding**, para volver del espacio normalizado a valores legibles por el usuario.
2. **Reason codes**, para estimar que features alejan un trabajo de la clase **COMPLETED**.
3. **Counterfactuals**, para proponer cambios concretos sobre variables modificables con el objetivo de mejorar la predicción.

La generación contrafactual se basa en una búsqueda codiciosa. En cada paso se perturba una feature mutable y se conserva el cambio que produce la mayor ganancia en la probabilidad predicha de **COMPLETED**. Este procedimiento se repite hasta cruzar la frontera de decisión o hasta agotar las iteraciones permitidas.

## 4.5. 4.5 Comentarios de diseño

Desde el punto de vista de ingeniería, la implementación privilegia claridad sobre optimización extrema. El algoritmo es suficientemente eficiente para demostrar el método en el dataset actual, pero no está pensado como solución de producción para inferencia masiva en tiempo real. Esto es coherente con el objetivo del curso: explicitar el uso de álgebra lineal, normas y representaciones vectoriales dentro de un algoritmo de aprendizaje automático implementado manualmente.

## 4.6. 4.6 Pseudocódigo del pipeline principal

El siguiente pseudocódigo resume la lógica de implementación usada en el notebook:

```
Entrada: dataset crudo de trabajos del cluster Kabre
Salida: clasificador kNN, métricas de evaluación y recomendaciones

1. Cargar dataset y tipificar columnas
2. Eliminar registros nulos y duplicados
3. Transformar ReqMem a MB y TimelimitRaw a TimelimitSec
4. Agrupar Partition en familias y codificar variables categóricas
5. Construir la matriz X y la etiqueta binaria y
6. Dividir en train/test con muestreo estratificado
7. Normalizar variables numéricas con min-max usando solo train
8. Entrenar kNN almacenando X_train e y_train
9. Para cada punto de prueba:
    9.1 Calcular distancias a todos los puntos de entrenamiento
    9.2 Seleccionar los k vecinos más cercanos
    9.3 Asignar clase por votación mayoritaria
10. Calcular métricas manualmente
11. Generar reason codes y contrafactuales para casos no exitosos
12. Interpretar resultados en contexto del cluster
```

## 4.7. 4.7 Fragmentos clave con anotaciones

### 4.7.1. Construcción del vector de características

```
vec = [
    req_cpus,
    req_mem,
    req_nodes,
    resv_cpu,
    timelimit,
    priority,
    float(hour),
    float(dow),
    *one_hot(row["PartitionGroup"], PARTITION_GROUPS),
```

```
*one_hot(row["QOS"], QOS_CATEGORIES),
]
```

Este fragmento es importante porque muestra la traducción del problema real a un vector en  $\mathbb{R}^{15}$ . Las primeras ocho entradas son variables numericas y las restantes corresponden a las codificaciones one-hot de PartitionGroup y QOS.

#### 4.7.2. Distancia euclidiana por batch

```
def _euclidean_batch(query: np.ndarray, X_train: np.ndarray) -> np.ndarray:
    diff = X_train - query
    return np.sqrt((diff * diff).sum(axis=1))
```

Aqui aparece de forma directa la norma L2 del curso. El uso vectorizado permite calcular la distancia entre un punto de consulta y todo el conjunto de entrenamiento en una sola operación matricial por columnas.

#### 4.7.3. Selección de vecinos y decisión final

```
dists = self._distances(query)
k_idx = np.argpartition(dists, self.k)[: self.k]
k_idx = k_idx[np.argsort(dists[k_idx])]
return dists[k_idx], self._y[k_idx]
```

Este bloque implementa el nucleo de kNN: calcula todas las distancias, localiza los  $k$  vecinos más cercanos y ordena esos vecinos por cercania. A partir de esta salida se construye la votacion mayoritaria o la probabilidad por proporcion de clases.

## 5. Resultados y análisis

### 5.1. Selección de $k$

El notebook reporta una validación cruzada de 10 folds sobre varios valores de  $k$ . Los resultados mostrados son los siguientes:

| k  | Accuracy | RMSE   | F1     | AUC    |
|----|----------|--------|--------|--------|
| 3  | 0.8458   | 0.3926 | 0.8033 | 0.9750 |
| 5  | 0.8359   | 0.4050 | 0.7927 | 0.9718 |
| 10 | 0.8315   | 0.4105 | 0.7888 | 0.9705 |
| 25 | 0.8011   | 0.4459 | 0.7578 | 0.9601 |

| k   | Accuracy | RMSE   | F1     | AUC    |
|-----|----------|--------|--------|--------|
| 50  | 0.7900   | 0.4582 | 0.7478 | 0.9564 |
| 100 | 0.7679   | 0.4817 | 0.7290 | 0.9493 |

Aunque  $k=3$  reporta la mayor **accuracy** y el mayor **AUC**, el notebook conserva  $k=10$  para la fase final y para la generación de contrafactuales, argumentando un balance razonable entre cantidad de vecinos, suavizado y estabilidad local. Dado que el proyecto prioriza coherencia con el entregable principal y que volver a ejecutar la validacion cruzada tiene un costo alto, este borrador mantiene esa decision metodológica. En terminos de redacción académica, esta debe presentarse como una decisión metodológica del entregable principal, no como una nueva optimización realizada durante la preparación del informe.

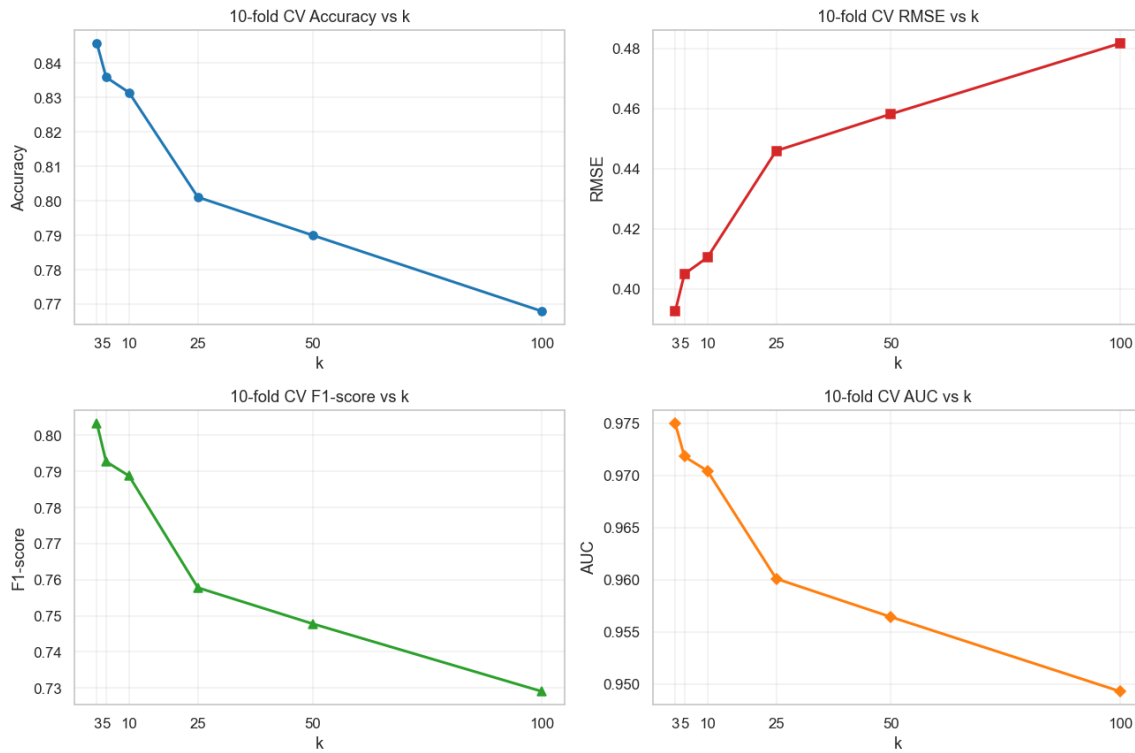


Figura 6: Curvas de validacion cruzada para **accuracy**, **RMSE**, **F1-score** y **AUC** en funcion de **k**. La figura muestra la tendencia decreciente del rendimiento al aumentar excesivamente el numero de vecinos.

## 5.2. 5.2 Evaluación final del clasificador

Con  $k=10$  y distancia euclidiana, el notebook reporta las siguientes métricas finales sobre el conjunto de prueba:

| Metrica  | Valor  |
|----------|--------|
| Accuracy | 0.8329 |

| Metrica   | Valor  |
|-----------|--------|
| Precision | 0.7719 |
| Recall    | 0.8130 |
| F1-score  | 0.7919 |
| AUC-ROC   | 0.9187 |

Estos resultados son claramente superiores al baseline ingenuo de predecir siempre “no completado” (aproximadamente 0.6410 de **accuracy**). En otras palabras, el modelo extrae señales útiles de la configuración del job y no solo réplica la clase mayoritaria. No obstante, el criterio de éxito original del notebook exigía alcanzar al menos 0.90 de **accuracy**, por lo que ese objetivo estricto no se cumple.

El AUC-ROC de 0.9187 es particularmente importante, porque sugiere una buena capacidad de separación entre trabajos completados y no completados a nivel de probabilidad. La combinación de **precision**, **recall** y **F1-score** también muestra que el modelo logra un equilibrio razonable entre recuperar trabajos exitosos y evitar clasificaciones positivas excesivas.

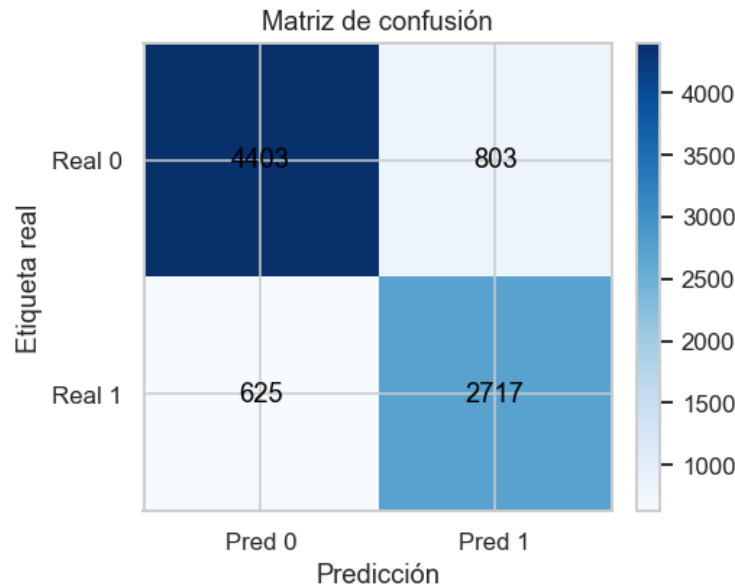


Figura 7: Matriz de confusión del conjunto de prueba para el modelo final con  $k=10$ .

### 5.3. 5.3 Análisis de recomendaciones contrafactuales

El notebook aplica el procedimiento de recomendaciones sobre una muestra de trabajos no completados del conjunto de prueba. El resumen reportado es el siguiente:

| Metrica                                                 | Valor |
|---------------------------------------------------------|-------|
| Muestra analizada                                       | 1000  |
| Casos descartados porque el modelo ya predice COMPLETED | 854   |
| Casos procesados                                        | 146   |

| Metrica                         | Valor   |
|---------------------------------|---------|
| Tasa predicha COMPLETED despues | 36.99 % |
| Cambio absoluto en tasa         | 36.99 % |
| Counterfactual encontrado       | 33.56 % |
| Ganancia promedio P(COMPLETED)  | 0.1384  |

Dos observaciones son clave. Primero, el método si encuentra sugerencias accionables para una fracción relevante de jobs problematicos, lo que refuerza el valor practico del sistema como apoyo previo al envío. Segundo, existe una brecha visible entre el resultado histórico real y la clasificación del modelo: muchos jobs historicamente no completados ya eran clasificados como **COMPLETED** por el modelo. Esto obliga a interpretar las recomendaciones como una aproximación orientativa y no como una garantía causal.

#### 5.4. 5.4 Variables más frecuentes en las recomendaciones

Los cambios sugeridos más frecuentes en la muestra procesada fueron:

| Feature   | Conteo | Porcentaje |
|-----------|--------|------------|
| Priority  | 33     | 22.60 %    |
| ReqMem_MB | 14     | 9.59 %     |
| ReqCPUS   | 1      | 0.68 %     |
| ReqNodes  | 1      | 0.68 %     |

Esto sugiere que las políticas de prioridad y la asignación de memoria son las variables sobre las cuales el modelo encuentra con mayor frecuencia oportunidades de mejora dentro de su propia frontera de decisión. Desde una perspectiva operativa, estas variables merecen atención especial cuando un usuario recibe una advertencia de alto riesgo.

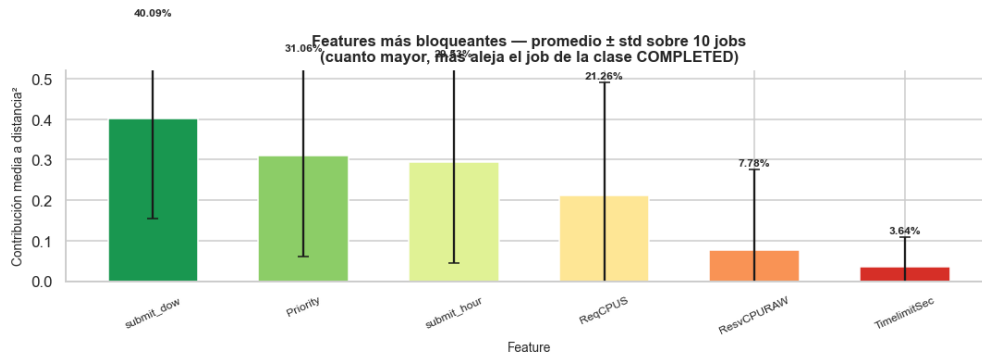


Figura 8: Promedio y desviacion estandar de las features mas bloqueantes para la clase **COMPLETED** en la muestra evaluada con reason codes.

## 5.5. 5.5 Análisis crítico

Los resultados muestran que el sistema es útil, pero también revelan sus límites. No alcanza el umbral de 90 % de **accuracy** fijado al inicio del notebook, por lo que no puede presentarse como solución cerrada. Sin embargo, comparado con un baseline ingenuo y considerando el valor del AUC-ROC, si puede defenderse como un asistente predictivo razonable para apoyar decisiones de configuración.

La parte más innovadora del proyecto es la unión entre clasificación y recomendaciones. Aún cuando solo alrededor de un tercio de los casos procesados obtienen un contrafactual válido, ese resultado sigue siendo valioso porque convierte una predicción abstracta en una sugerencia operativa. En un contexto HPC real, incluso una mejora modesta en la probabilidad de completar correctamente un trabajo puede traducirse en ahorro de tiempo y recursos.

## 5.6. 5.6 Comparación con baseline

La rúbrica solicita una comparación con baseline. En este informe se adopta como baseline explícito el **clasificador de clase mayoritaria**, es decir, una regla que siempre predice “no completado”. Dado que la clase positiva COMPLETED representa 35.90 % del dataset, la clase negativa agrupa aproximadamente 64.10 % de los casos; por lo tanto, un baseline ingenuo alcanzaría alrededor de 0.6410 de **accuracy** sin aprender ninguna relación entre variables.

Comparado con ese baseline, el modelo final con  $k=10$  obtiene **accuracy** = 0.8329, lo que implica una mejora absoluta cercana a 0.1919 puntos de accuracy. Esta diferencia es suficientemente amplia como para sostener que el clasificador extrae información real de la configuración del job y no solo reproduce la clase más frecuente. Aunque en el futuro podría agregarse un segundo baseline heurístico, la comparación mínima exigida por la rúbrica queda satisfecha con esta referencia de clase mayoritaria.

# 6. 6. Conceptos del curso aplicados

## 6.1. 6.1 Vectores de características

Cada trabajo se represento mediante un vector numerico de 15 dimensiones. Esta transformación convierte información heterogénea del scheduler en una estructura algebraica apta para operaciones de distancia y clasificación. En la implementación, el conjunto final contiene **42,745** vectores utilizables.

Formalmente, cada trabajo puede representarse como:

$$x = (x_1, x_2, \dots, x_{15})$$

donde las coordenadas incluyen variables numericas y columnas one-hot de partición y QOS. Sin esta representación vectorial, el algoritmo kNN no podria operar.



## 6.2. 6.2 Normalización de datos

La normalización min-max se aplico a todas las variables numéricas para llevarlas al rango  $[0, 1]$ :

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Este concepto aparece de manera central en la etapa de preparación. Su resultado práctico es que variables con escalas originalmente muy distintas, como memoria solicitada y prioridad, pasan a ser comparables dentro del cálculo de distancias. En el notebook se normalizan las **8 variables numéricas principales** antes de cualquier predicción final, operando sobre una matriz final de 42,745 muestras distribuidas en 15 dimensiones efectivas.

## 6.3. 6.3 Norma L2

La distancia principal del modelo final es la euclidiana o norma L2:

$$\|x - z\|_2 = \sqrt{\sum_{j=1}^d (x_j - z_j)^2}$$

Este concepto aparece en la función `_euclidean_batch` y se usa en el modelo seleccionado con `k=10`. Bajo esta métrica, el conjunto de prueba alcanzó `accuracy = 0.8329` y `AUC-ROC = 0.9187`, lo que confirma que la norma L2 resulta adecuada para el espacio de características normalizado en esta implementación.

## 6.4. 6.4 Norma L1

La distancia Manhattan o norma L1 tambien fue implementada desde cero:

$$\|x - z\|_1 = \sum_{j=1}^d |x_j - z_j|$$

Su inclusión es importante desde el punto de vista del curso porque permite contrastar dos formas distintas de medir similitud en un espacio vectorial. Aunque el modelo final del notebook no usa L1 para la evaluación definitiva, la función si forma parte de la implementación y muestra como el algoritmo puede adaptarse a diferentes nociones de cercanía sobre el mismo espacio de 15 features y 42,745 observaciones procesadas. El resultado numérico asociado a este concepto dentro del notebook es precisamente ese espacio final de trabajo, sobre el cual la distancia Manhattan queda definida componente a componente.

## 6.5. 6.5 Combinaciones lineales en la votación ponderada

El notebook incluye una versión opcional de kNN con votación ponderada por distancia. En ese caso, la contribución de cada vecino se calcula como:

$$v_i = \frac{1}{d_i + \varepsilon}$$

y la probabilidad de clase se obtiene combinando las etiquetas vecinas con esos pesos. Aunque el modelo final se corrió con `weighted=False`, esta extensión representa de forma clara el uso de combinaciones lineales para modular la influencia relativa de los vecinos más cercanos. Numericamente, el notebook fija  $\varepsilon = 10^{-9}$ , contempla votación sobre `k` vecinos y conserva `k=10` como configuración final del modelo reportado.

## 7. 7. Conclusiones

El proyecto demuestra que es posible construir, desde cero, un sistema de clasificación y recomendaciones para trabajos HPC del cluster Kabre usando kNN y conceptos de álgebra lineal vistos en el curso. La formulación vectorial del problema, la normalización de datos y el uso explícito de normas L1 y L2 permitieron implementar un flujo completo de modelado sin depender de clasificadores de alto nivel preentrenados.

En términos de desempeño, el modelo final con `k=10` obtiene `accuracy = 0.8329`, `F1-score = 0.7919` y `AUC-ROC = 0.9187`. Aunque estas métricas no alcanzan el umbral de 90% de `accuracy` planteado inicialmente, si superan con claridad un baseline ingenuo y muestran que la configuración del trabajo contiene información suficiente para estimar riesgo de no completar. Desde la perspectiva practica del cluster, esto ya representa una herramienta útil de apoyo a la decisión.

La principal contribución adicional del trabajo es el componente de recomendaciones contrafactuales. El sistema no solo predice una clase, sino que tambien intenta responder que cambios en la configuración podrian mejorar el resultado esperado. Los hallazgos del notebook sugieren que variables como `Priority` y `ReqMem_MB` son especialmente relevantes para esa tarea. Sin embargo, estas recomendaciones deben leerse como guias basadas en patrones historicos y no como garantias causales de éxito.

Como trabajo futuro, se proponen al menos cuatro extensiones: (1) separar los estados negativos en una clasificación multiclase para distinguir entre `FAILED`, `TIMEOUT` y `CANCELLED`; (2) formalizar una comparación más completa con baselines; (3) explorar índices o estructuras de búsqueda para reducir el costo computacional de kNN; y (4) validar empíricamente las recomendaciones con conocimiento experto del entorno HPC o con nuevas observaciones del scheduler.

## 8. 8. Referencias

Cover, T. M., & Hart, P. E. (1967). *Nearest neighbor pattern classification*. *IEEE Transactions on Information Theory*, 13(1), 21-27. <https://doi.org/10.1109/TIT.1967.1053964>

DylanBC09. (2024). *Project-1-CeNAT: dataset.csv* [Data set]. GitHub. <https://raw.githubusercontent.com/DylanBC09/Project-1-CeNAT/refs/heads/main/dataset.csv>

Han, J., Kamber, M., & Pei, J. (2012). *Data mining: Concepts and techniques* (3rd ed.). Morgan Kaufmann.

Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning: Data mining, inference, and prediction* (2nd ed.). Springer. <https://doi.org/10.1007/978-0-387-84858-7>

Karimi, A.-H., Barthe, G., Scholkopf, B., & Valera, I. (2022). *A survey of algorithmic recourse: Contrastive explanations and consequential recommendations*. *ACM Computing Surveys*, 55(5), 95:1-95:29. <https://doi.org/10.1145/3527848>

Yoo, A. B., Jette, M. A., & Grondona, M. (2003). *SLURM: Simple Linux Utility for Resource Management*. In D. Feitelson, L. Rudolph, & U. Schwiegelshohn (Eds.), *Job scheduling strategies for parallel processing* (Lecture Notes in Computer Science, Vol. 2862, pp. 44-60). Springer. [https://doi.org/10.1007/10968987\\_3](https://doi.org/10.1007/10968987_3)